Build database files

Ben Boeckel, Daniel Ruoso
<ben.boeckel@kitware.com, druoso@bloomberg.net>
version P2977R2, 2024-10-14

Table of Contents

Document number	ISO/IEC/JTC1/SC22/WG21/P2977R2
Date	2024-10-14
Reply-to	Ben Boeckel, Daniel Ruoso, ben.boeckel@kitware.com, druoso@bloomberg.net
Audience	EWG (Evolution), SG15 (Tooling)

1. Abstract

Build tools need to be able to manage modules for projects that do not use the same compiler as the tool. However, compiled module formats are generally not compatible between

compilers (or even compiler versions) or tools which may need to analyze modules for their own purposes (e.g., static analyzers). These tools need to be able to compile module interfaces in their own format with an understanding that corresponds to the actual build and therefore need to know what flags are relevant in order to make analogous compiled modules as the main build.

2. Changes

2.1. R2 (Wroclaw)

This revision incorporates updates based on experience in implementing the information described in R1. This includes:

- adding a language indicator to support specifying more than just C++
- set names may now be `null`; these sets may not be visible from any other set
- move baseline arguments to the set rather than each translation unit

2.2. R1 (Tokyo)

This revision incorporates updates based on experience in implementing the information described in R0. This includes:

- including the baseline arguments for a module set;
- adding a "family name" to help draw associations between related sets;
- including BMI paths for provided modules;
- including information beyond just module-providing sources.

This revision also selects the "Standalone" representation from the first revision as its proposed format. Given the expanded scope, referring to a compile commands database turns out to be significant duplication and not worthwhile.

2.3. R0 (Initial)

Initial paper.

3. Contents

In order to analyze a build with modules in it, a tool needs to know:

- the module interface source;
- the language the source uses;
- the names of modules that it requires;
- the set of sources which may provide modules which are required;
- the visibility of the module within the set of sources;

- the "baseline arguments" for a set to apply to modules from visible sets;
- the set of "local preprocessor arguments" used during the build when processing the source; and
- the working directory of the compilation.

Given this, a tool may traverse the dependency graph through the set of sources information for a given source in order to generate the appropriate module artifacts for whatever analysis is being performed.

3.1. Interface source

The interface source path is required so that the tool understands what consitutes the module itself.

3.2. Language

The source may be in another source language. Not all of the world is C++ and other languages (e.g., Objective-C++) may support consuming C++ modules. Additionally, build databases may contain entries for compilation of C, Fortran, etc. Standardized values are:

- `c`
- `c++`
- `fortran`
- `objective-c`
- `objective-c++`

Languages not listed here may be specified with an `ext:` prefix and standardized as a new `minor` version.

3.3. Provided modules

The set of provided modules and their BMIs are required so that imports of the module may know which source provides each module to satisfy an import.

3.4. Required modules

Modules imported by the module also need to be know in order to prepare satisfaction of the contained `import` statements.

3.5. Visible sets

Translation units are described in terms of "sets". Only modules that are provided by members of sets which are visible of the translation unit's owning set may be used to satisfy `import` requests within the current module.

A set is implicitly visible to itself with the additional power to also use modules provided by `private` translation units.

3.6. Visibility

Translation units that are part of a set may be marked as "private" to indicate that they are not eligible for use by other sets. For example, the contained symbols might not be accessible at runtime (using `-fvisibility=hidden` or a lack of `__declspec` decorations). They are specified in order to given more useful diagnostics if they are mentioned from other sets and to indicate usage within the set itself.

3.7. Baseline arguments

These arguments are used for a set when applied to any imported modules in order to create compatible BMI files. Tooling may need to "translate" flags for the compiler in use (where the flags come from) for itself.

3.8. Local arguments

These arguments are required to be used to create the module in a corresponding way. Tooling may need to "translate" flags for the compiler in use (where the flags come from) for itself.

3.9. Working directory

Tooling interprets relative paths differently based on the current working directory. This field is specified so that tooling may agree with the compiler as needed.

4. Sets of translation units

Each set contains a collection of translation units that "belong" together. They should have some connection with each other (e.g., part of the same library artifact), but there are no defined semantics of the format itself. Sets are unique by their `name` field but may indicate a "family name" as well. Sets which share a family name are related (e.g., by having the same source files). Each set then represents a single "view" on the sources that create separate, but incompatible, BMIs for consumers to use. They may differ based on local arguments or baseline arguments depending on whether the BMIs are needed for different configurations (e.g., "release" versus "debug") or consumers (e.g., incompatible flags).

A set may only list a single member of a family in its list of visible sets. If multiple family members of a family could be visible, it would be ambiguous which family member provides modules from the common sources.

4.1. Tooling

Tools which consume build databases may have different views on what constitutes an incompatible BMI. Tools may need to create their own new family members to support combinations of local arguments and baseline arguments used during the actual build.

4.2. Unnamed sets

Sets may also be unnamed (i.e., `null`). Such sets are not eligible to be visible from any other set for the purposes of modules and are considered separate from any other unnamed set.

5. Representation

There are a few potential ways in which this information may be represented within a build tree. Here, a few possible representations are presented.

5.1. Standalone

The first option is for a standalone database which contains all of the relevant information. It might be in separate files (see below) and later combined into a single database for convenience.

A benefit of this is that the content could be reused for the installation rather than just the build tree (as a compilation database doesn't make sense for installations).

5.2. Cross-reference with Compile Commands Database

Another way would be for the module database to be used in conjunction with a compilation database [json-cdb]. This would help to reduce duplication, but would require tooling to manually perform joins on the two databases to get all of the required information.

The main issue with this approach is that the most reliable way of correlating the module database with the compilation database relies on an optional value (`output`) as a single source might participate in a build graph more than once with different flags (e.g., building release and debug or static and shared variants at the same time).

It would also likely involve adopting the compilation database from the Clang team and into ISO with the additional enhancements specified for modules.

5.3. Share with Compile Commands Database

Another alternative would be to split the information into parts that could be shared with the compilation database (such as the local preprocessor flags) and the module-specific information (such as module sets and their dependencies) refer to this shared information as well.

This approach would not require that the compilation database be adopted into ISO as it would just also have pointers to the shared portion (though the rationale for the split may be awkward).

6. JSON Schema

This paper selects the "Standalone" solution in order to keep it simpler as the compilation database would be largely duplicated with the additional information.

▶ JSON Schema for the format

7. Availability

Generally, these module compilation databases must be created during the build itself. This is because the set of module names in a build are not necessarily known until the build is underway.

However, this is not a new problem as the compilation database can refer to the compilation of generated sources which do not exist until the build has completed some of its work.

Build systems should offer mechanisms to combine module compilation databases together into combined files in well-known locations so that consuming tools do not need to search for relevant files and have a reliable way to make sure that the information is consistent across the entire file. This would be provided by relying on standard features of build systems to update outputs when inputs change and the appropriate dependency information provided.

8. Combining

Combining two module compilation databases is as trivial as ensuring the version information is consistent and appending each file's `sets` arrays into a single array and writing out the combined information.

9. Versioning

There are two properties with integer values in the top-level JSON object of the format: `version` and `revision`. The `version` property is required and if `revision` is not provided, it can be assumed to be `0`. These indicate the version of the information available in the format itself and what features may be used.

The `version` integer is incremented when semantic information required for a correct interpretation is different than the previous version. When the `version` is incremented, the `revision` integer is reset to `0`.

The `revision` integer is incremented when the semantic information of the format is the same as the previous revision of the same version, but it may include additionally specified information or use an additionally specified format for the same information. For example, adding a `modification_time` or `input_hash` field may be helpful in some cases, but is not required to understand the dependency graph. Such an addition would cause an increment of the `revision` value.

The version specified in this document is:

Version fields for this specification

```
{
  "version": 1,
  "revision": 0
}
```

10. References

- [json-cdb] JSON Compilation Database Format Specification. https://clang.llvm.org/docs/JSONCompilationDatabase.html.

Version P2977R2
Last updated 2024-10-15 10:20:19 EDT